

Principio de Inversión de Dependencia (DIP) y Principio de Segregación de Interfaces (ISP)

Recordatorio

1 Clase, objeto e interface en lenguaje sencillo

Una clase describe cómo será un objeto. Un objeto es una instancia concreta creada a partir de una clase. Una interface, en cambio, funciona como un contrato: indica qué operaciones deben existir, pero no obliga a una única forma de realizarlas.

Ejemplo mínimo de interface

```
public interface Saludador {  
    void saludar();  
}  
  
public class SaludadorEspanol implements Saludador {  
    @Override  
    public void saludar() {  
        System.out.println("Hola");  
    }  
}
```

2 ¿Qué es una dependencia?

Una clase A depende de una clase B cuando A necesita a B para poder realizar su trabajo. Tener dependencias no es malo; el problema aparece cuando A está amarrada a una implementación concreta que no puede cambiarse fácilmente.

Dependencia directa

```
public class ReporteService {  
    private ImpresoraHP impresora = new ImpresoraHP();  
  
    public void imprimirReporte() {  
        impresora.imprimir("Reporte mensual");  
    }  
}
```

Pregunta al aula: Si mañana la empresa cambia la impresora HP por una Epson, ¿qué clase debemos modificar? La respuesta es ReporteService. Esa rigidez es la pista que necesitamos para hablar de DIP.

3. Principio de Inversión de Dependencia (DIP)

3.1 Definición

DIP significa Dependency Inversion Principle. En forma académica, los módulos de alto nivel no deben depender directamente de módulos de bajo nivel; ambos deben depender de abstracciones. Además, las abstracciones no deben depender de los detalles: los detalles deben depender de las abstracciones.

Traducción para principiantes: “La clase importante no debería decir: necesito exactamente esta clase. Debería decir: necesito algo que cumpla este contrato.”

3.2 Analogía del enchufe

Un equipo eléctrico se conecta a un enchufe estándar. El equipo no necesita conocer cómo la empresa eléctrica genera la energía. Si mañana cambia la fuente de energía, el equipo sigue usando el mismo contrato de conexión. En programación, la interface cumple un papel similar.

3.3 Ejemplo 1 – código que NO aplica DIP

Supongamos que una tienda registra una compra y debe enviar una notificación por correo. La primera versión funciona, pero CompraService crea directamente un NotificadorEmail.

```
public class NotificadorEmail {
    public void enviar(String mensaje) {
        System.out.println("EMAIL: " + mensaje);
    }
}

public class CompraService {
    private NotificadorEmail notificador = new NotificadorEmail();

    public void realizarCompra(double monto) {
        System.out.println("Compra realizada por S/ " + monto);
        notificador.enviar("Su compra fue registrada");
    }
}
```

¿Qué problema existe?

- CompraService conoce la clase concreta NotificadorEmail.
- Si ahora queremos SMS, WhatsApp o notificación por consola, debemos modificar CompraService.
- La regla de negocio “realizar compra” queda mezclada con un detalle técnico “cómo notificar”.
- Mientras más detalles concretos tenga CompraService, más difícil será mantenerlo.

Idea clave: El programa funciona. DIP no corrige un “error de sintaxis”; corrige un problema de diseño que aparece cuando el software cambia.

3.4 Refactor paso a paso para aplicar DIP

Paso 1. Crear una abstracción

```
public interface Notificador {
    void enviar(String mensaje);
}
```

La interface no sabe si el mensaje irá por correo, SMS o cualquier otro canal. Solo define lo que CompraService necesita: poder enviar un mensaje.

Paso 2. Hacer que los detalles implementen la interface

```
public class NotificadorEmail implements Notificador {
    @Override
    public void enviar(String mensaje) {
        System.out.println("EMAIL: " + mensaje);
    }
}

public class NotificadorSMS implements Notificador {
    @Override
    public void enviar(String mensaje) {
        System.out.println("SMS: " + mensaje);
    }
}
```

Paso 3. Hacer que la clase de alto nivel dependa de la interface

```
public class CompraService {  
    private Notificador notificador;  
  
    public CompraService(Notificador notificador) {  
        this.notificador = notificador;  
    }  
  
    public void realizarCompra(double monto) {  
        System.out.println("Compra realizada por S/ " + monto);  
        notificador.enviar("Su compra fue registrada");  
    }  
}
```

CompraService ya no crea un NotificadorEmail. Solo recibe “algo que sea Notificador”. Esto se llama inyección de dependencia por constructor. Para esta sesión, esta es la forma más simple y clara de inyección de dependencias.

Paso 4. Elegir la implementación desde Main

```
public class Main {  
    public static void main(String[] args) {  
        Notificador notificador = new NotificadorEmail();  
        CompraService compraService = new CompraService(notificador);  
  
        compraService.realizarCompra(120.50);  
    }  
}
```

Prueba en vivo: Cambie solamente new NotificadorEmail() por new NotificadorSMS(). CompraService no cambia. Ese es el beneficio observable de DIP.

Ejemplo clásico – una interface demasiado grande

```
public interface MaquinaOficina {
    void imprimir();
    void escanear();
    void enviarFax();
}

public class ImpresoraBasica implements MaquinaOficina {
    @Override
    public void imprimir() {
        System.out.println("Imprimiendo...");
    }

    @Override
    public void escanear() {
        throw new UnsupportedOperationException("No escanea");
    }

    @Override
    public void enviarFax() {
        throw new UnsupportedOperationException("No tiene fax");
    }
}
```

¿Qué está mal?

- ImpresoraBasica solo sabe imprimir.
- La interface la obliga a declarar escanear() y enviarFax().
- Los métodos que lanzan “no soportado” son una señal clara de que el contrato es demasiado grande.
- Una clase cliente que solo necesita imprimir también termina viendo operaciones que no le interesan.

5.3 Aplicar ISP paso a paso

Paso 1. Dividir la interface

```
public interface Imprimible {
    void imprimir();
}

public interface Escaneable {
    void escanear();
}

public interface Faxable {
    void enviarFax();
}
```

Paso 2. Implementar solamente lo necesario

```
public class ImpresoraBasica implements Imprimible {
    @Override
    public void imprimir() {
        System.out.println("Imprimiendo...");
    }
}
```

Paso 3. Una clase completa puede implementar varias interfaces

```
public class Multifuncional implements Imprimible, Escaneable, Faxable {  
    @Override  
    public void imprimir() {  
        System.out.println("Imprimiendo...");  
    }  
  
    @Override  
    public void escanear() {  
        System.out.println("Escaneando...");  
    }  
  
    @Override  
    public void enviarFax() {  
        System.out.println("Enviando fax...");  
    }  
}
```

Idea clave: ISP no significa “una interface por cada método” de forma automática. Significa diseñar contratos coherentes con las necesidades reales de los clientes.